

First-Order Logic Inference

Lecture 16

CS 4880

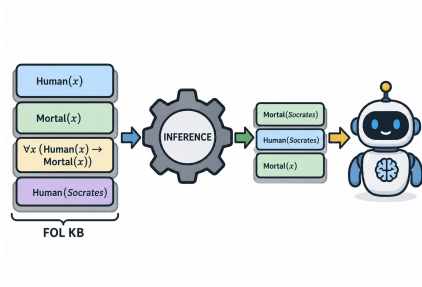
Kirk Duran

Representation Is Only Half the Job

Last time we gave the agent a richer language:

- ▶ objects and variables;
- ▶ predicates and relations;
- ▶ functions and nested terms;
- ▶ universal and existential rules.

But a knowledge base is useful only if the agent can *derive new facts* from those general rules.



Key idea

First-order inference asks how general symbolic rules become concrete conclusions about particular objects.

The Human Step We Need to Automate

Suppose the knowledge base contains

$$\forall x \text{ Human}(x) \rightarrow \text{Mortal}(x)$$

and

$$\text{Human}(\text{Socrates}).$$

We immediately conclude

$$\text{Mortal}(\text{Socrates}).$$

What did your brain silently do?

It noticed that the variable x in the rule can stand for *Socrates*.

Key idea

The core computational problem is matching variable-containing expressions to specific terms.

Three Pieces of First-Order Inference

1. Substitution

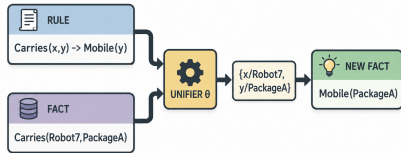
Replace variables with terms in a consistent way.

2. Unification

Find substitutions that make expressions match.

3. Inference

Apply matched rules to derive new knowledge.



Key idea

Unification is the bridge between representation and automated reasoning.

Substitution: Giving Variables Concrete Values

A **substitution** is a mapping from variables to terms.

$$\theta = \{x/Socrates\}$$

means “replace x with *Socrates*.”

Applying θ to a sentence:

$$Human(x)\theta = Human(Socrates).$$

Key idea

A substitution does not change the rule's meaning; it creates one particular instance of a general pattern.

Substitutions Can Bind Several Variables

Consider

$$\theta = \{x/Alice, y/Room214\}.$$

Then

$$In(x, y)\theta = In(Alice, Room214).$$

and

$$Connected(y, Hallway2)\theta = Connected(Room214, Hallway2).$$

Key idea

The same substitution must be applied consistently everywhere the bound variable appears.

Substitution Works Inside Nested Terms

Suppose

$$\theta = \{x/Alice\}.$$

Then

$$SupervisorOf(x)\theta = SupervisorOf(Alice)$$

and

$$Knows(x, SupervisorOf(x))\theta = Knows(Alice, SupervisorOf(Alice)).$$

Key idea

Substitution recursively replaces variables wherever they occur inside a term or sentence.

Ground Expressions Have No Variables

Contains variables

$Carries(r, p)$
 $In(r, LocationOf(p))$

Ground

$Carries(Robot7, PackageA)$
 $In(Robot7, Room214)$

Key idea

A ground sentence refers only to specific terms; no variable bindings remain to be chosen.

Pattern Matching Is the Simplest Case

A rule may contain a pattern such as

$$\textit{Carries}(r,p).$$

A fact may be

$$\textit{Carries}(\textit{Robot7}, \textit{PackageA}).$$

Matching discovers

$$\theta = \{r/\textit{Robot7}, p/\textit{PackageA}\}.$$

Key idea

Pattern matching asks which variable bindings make a template identical to a concrete expression.

A Match Can Fail Immediately

Try to match

$$\textit{In}(\textit{Robot7}, x)$$

with

$$\textit{In}(\textit{Robot9}, \textit{Room214}).$$

The first arguments are different constants:

$$\textit{Robot7} \neq \textit{Robot9}.$$

No substitution for x can repair that mismatch.

Key idea

Variables can be bound; distinct constants cannot simply be rewritten to match each other.

Unification Generalizes Pattern Matching

Definition

Unification finds a substitution that makes two first-order expressions syntactically identical.

Example:

$Knows(Alice, x)$

and

$Knows(y, Bob)$

unify under

$$\theta = \{y/Alice, x/Bob\}.$$

Key idea

Both sides may contain variables; unification solves the bindings jointly.

Unification Must Respect Structure

To unify

$$\textit{Parent}(x, y)$$

with

$$\textit{Parent}(\textit{Alice}, \textit{Bob}),$$

we compare corresponding pieces:

$$x \leftrightarrow \textit{Alice}, \quad y \leftrightarrow \textit{Bob}.$$

So

$$\theta = \{x/\textit{Alice}, y/\textit{Bob}\}.$$

Key idea

Predicate names, function names, arity, and argument positions must all line up.

Repeated Variables Create Constraints

Can these expressions unify?

Likes(x , x)

Likes(*Alice*, *Bob*)

The first argument requires

$x = \textit{Alice}$,

while the second requires

$x = \textit{Bob}$.

If $\textit{Alice} \neq \textit{Bob}$, no consistent substitution exists.

Key idea

One variable must have one consistent binding throughout the expression.

Functions Unify by Their Internal Structure

Consider

$$\textit{Knows}(x, \textit{MotherOf}(x))$$

and

$$\textit{Knows}(\textit{Alice}, \textit{MotherOf}(\textit{Alice})).$$

The first arguments suggest

$$x = \textit{Alice}.$$

The second arguments then become identical automatically.

$$\theta = \{x/\textit{Alice}\}.$$

Key idea

Compound terms unify recursively: first the outer function, then its arguments.

Nested Terms Can Force More Bindings

Unify

$$\textit{Knows}(x, \textit{SupervisorOf}(y))$$

with

$$\textit{Knows}(\textit{Alice}, \textit{SupervisorOf}(\textit{Bob})).$$

Matching the structure yields

$$x = \textit{Alice}, \quad y = \textit{Bob}.$$

Therefore

$$\theta = \{x/\textit{Alice}, y/\textit{Bob}\}.$$

Key idea

Unification decomposes complex expressions until it reaches variable-term bindings or a contradiction.

Some Structures Can Never Unify

Different predicate

Likes(*Alice*, *x*)

Knows(*Alice*, *Bob*)

Fail: *Likes* $\neq$ *Knows*.

Different function

OwnerOf(*x*)

SupervisorOf(*Alice*)

Fail: function symbols differ.

Key idea

Unification changes variable bindings, not the fixed symbolic structure of the language.

The Occurs Check Prevents Circular Terms

Suppose we try to unify

x

with

$ParentOf(x).$

Naively binding $x/ParentOf(x)$ would make x contain itself forever:

$$x = ParentOf(ParentOf(ParentOf(\dots))).$$

A standard unification algorithm rejects this with the **occurs check**.

Key idea

A variable cannot be bound to a term that already contains that same variable.

The Most General Unifier

Often many substitutions can make expressions match.

For

$$\text{Likes}(x, \text{IceCream})$$

and

$$\text{Likes}(\text{Alice}, y),$$

the useful substitution is

$$\theta = \{x/\text{Alice}, y/\text{IceCream}\}.$$

A **most general unifier (MGU)** makes only the bindings necessary to force the match.

Key idea

The MGU preserves as much freedom as possible instead of inventing unnecessary commitments.

Why “Most General” Matters

Suppose

$$Knows(x, y)$$

must unify with

$$Knows(Alice, z).$$

One MGU is

$$\theta = \{x/Alice, y/z\}.$$

We do *not* need to decide what z is yet.

Key idea

Keeping unresolved variables unresolved lets the result participate in later inference without losing valid possibilities.

Unification as an Algorithm

High-level idea

```
UNIFY(x, y,  $\theta$ )  
  if  $\theta$  is failure: return failure  
  if  $x = y$ : return  $\theta$   
  if x is a variable: UNIFY_VAR(x, y,  $\theta$ )  
  if y is a variable: UNIFY_VAR(y, x,  $\theta$ )  
  if both are compound with same operator:  
    unify corresponding arguments recursively  
  else: return failure
```

Key idea

Unification is a recursive structural comparison that accumulates variable bindings as it descends.

Worked Unification: Start with the Outer Structure

Unify

$Knows(x, SupervisorOf(y))$

with

$Knows(Alice, SupervisorOf(Bob)).$

Both use the predicate *Knows* with two arguments, so compare arguments pairwise:

x with *Alice*

and

$SupervisorOf(y)$ with $SupervisorOf(Bob).$

Key idea

Matching the outer symbol lets the algorithm safely recurse into corresponding arguments.

Worked Unification: Accumulate Bindings

First pair:

x with *Alice*

gives

$$\theta_1 = \{x/Alice\}.$$

Second pair has matching function symbol *SupervisorOf*, so compare

y with *Bob*.

This extends the substitution:

$$\theta_2 = \{x/Alice, y/Bob\}.$$

Key idea

The substitution grows as constraints are discovered.

Worked Unification: Verify the Result

Apply

$$\theta = \{x/Alice, y/Bob\}$$

to the first expression:

$$\begin{aligned} & \textit{Knows}(x, \textit{SupervisorOf}(y))\theta \\ &= \textit{Knows}(\textit{Alice}, \textit{SupervisorOf}(\textit{Bob})). \end{aligned}$$

It is now identical to the second expression.

Key idea

A candidate unifier is correct exactly when applying it makes the two expressions the same.

From Matching to Inference

Propositional modus ponens says

$$P, \quad P \rightarrow Q \quad \Longrightarrow \quad Q.$$

First-order rules contain variables, so we need to match the premises to known facts before firing the rule.

Key idea

Generalized Modus Ponens adds unification to the familiar rule “premises imply conclusion.”

Generalized Modus Ponens

Suppose the rule is

$$P_1 \wedge P_2 \wedge \cdots \wedge P_n \rightarrow Q.$$

If a substitution θ makes every premise match known facts, infer

$$Q\theta.$$

Definition

Generalized Modus Ponens applies a first-order rule after unifying its premises with facts in the knowledge base.

Key idea

Unification chooses the instance; the inference rule produces the conclusion.

Classic Example: King + Greedy $\Rightarrow$ Evil

Rule:

$$King(x) \wedge Greedy(x) \rightarrow Evil(x).$$

Facts:

$$King(John), \quad Greedy(John).$$

Unifier:

$$\theta = \{x/John\}.$$

Therefore:

$$Evil(John).$$

Key idea

One general rule can fire for any object whose facts satisfy its premises.

Campus Robot Rule

Suppose the robot uses

$$\textit{Robot}(r) \wedge \textit{Carries}(r,p) \wedge \textit{Fragile}(p) \rightarrow \textit{MoveSlowly}(r).$$

The KB contains

$$\begin{aligned} &\textit{Robot}(\textit{Robot7}), \\ &\textit{Carries}(\textit{Robot7}, \textit{PackageA}), \\ &\textit{Fragile}(\textit{PackageA}). \end{aligned}$$

Matching all three premises gives

$$\theta = \{r/\textit{Robot7}, p/\textit{PackageA}\}.$$

Key idea

Multi-premise rules require one substitution that satisfies the premises consistently together.

The Rule Fires with the Same Substitution

Apply

$$\theta = \{r/Robot7, p/PackageA\}$$

to the conclusion:

$$MoveSlowly(r)\theta = MoveSlowly(Robot7).$$

The new fact can now be added to the knowledge base.

Key idea

The variable bindings discovered while matching the premises are reused to instantiate the conclusion.

Forward Chaining Still Works—Now with Unification

1. Start with known facts.
2. Find a rule whose premises can unify with facts in the KB.
3. Apply the resulting substitution to the conclusion.
4. Add the new fact if it is not already known.
5. Repeat until the query is found or no new facts appear.

Key idea

Propositional forward chaining becomes first-order forward chaining by replacing exact premise matching with unification.

Forward Chaining Example: Initial Knowledge

Facts:

Robot(Robot7)

Carries(Robot7, PackageA)

Fragile(PackageA)

Rules:

$Robot(r) \wedge Carries(r, p) \wedge Fragile(p) \rightarrow MoveSlowly(r)$

$MoveSlowly(r) \rightarrow SafeCarrier(r)$

Key idea

Forward chaining asks: "Given what I know now, what new facts can I derive?"

Forward Chaining: First Rule Fires

The first rule matches under

$$\theta = \{r/Robot7, p/PackageA\}.$$

Infer

$$MoveSlowly(Robot7).$$

The knowledge base grows:

$$KB \leftarrow KB \cup \{MoveSlowly(Robot7)\}.$$

Key idea

New conclusions become facts that may enable additional rules.

Forward Chaining: The New Fact Triggers Another Rule

Now the rule

$$\textit{MoveSlowly}(r) \rightarrow \textit{SafeCarrier}(r)$$

matches the newly derived fact

$$\textit{MoveSlowly}(\textit{Robot7}).$$

with

$$\theta = \{r/\textit{Robot7}\}.$$

Therefore infer

$$\textit{SafeCarrier}(\textit{Robot7}).$$

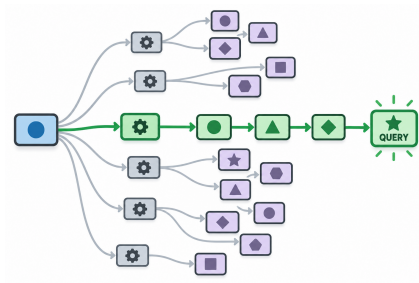
Key idea

Forward chaining propagates consequences outward from the currently known data.

Forward Chaining Can Generate Many Irrelevant Facts

If the KB has hundreds of rules, forward chaining may derive many facts unrelated to the question we actually care about.

- ▶ data-driven;
- ▶ systematic;
- ▶ useful when many consequences matter;
- ▶ potentially wasteful for one narrow query.



Key idea

This is the same recurring AI problem: computation is cheap only when we focus it on what matters.

Backward Chaining Starts from the Query

Suppose the query is

$$\text{SafeCarrier}(\text{Robot7})?$$

Rather than deriving everything, ask:

What rule could prove this?

$$\text{MoveSlowly}(r) \rightarrow \text{SafeCarrier}(r)$$

Unifying the conclusion with the query gives

$$\theta = \{r/\text{Robot7}\}.$$

The new subgoal is

$$\text{MoveSlowly}(\text{Robot7})?$$

Key idea

Backward chaining turns a goal into the premises that would be sufficient to prove it.

Backward Chaining Expands the Subgoal

To prove

$$\textit{MoveSlowly}(\textit{Robot7}),$$

use

$$\textit{Robot}(r) \wedge \textit{Carries}(r, p) \wedge \textit{Fragile}(p) \rightarrow \textit{MoveSlowly}(r).$$

Matching the conclusion requires

$$r = \textit{Robot7}.$$

This creates three subgoals:

$$\textit{Robot}(\textit{Robot7})?$$

$$\textit{Carries}(\textit{Robot7}, p)?$$

$$\textit{Fragile}(p)?$$

Key idea

Variables may remain unresolved until later subgoals provide enough information to bind them.

A Later Subgoal Can Finish the Binding

The KB contains

Carries(Robot7, PackageA).

Unifying it with

Carries(Robot7, p)

gives

$p = \text{PackageA}.$

Now the remaining goal becomes

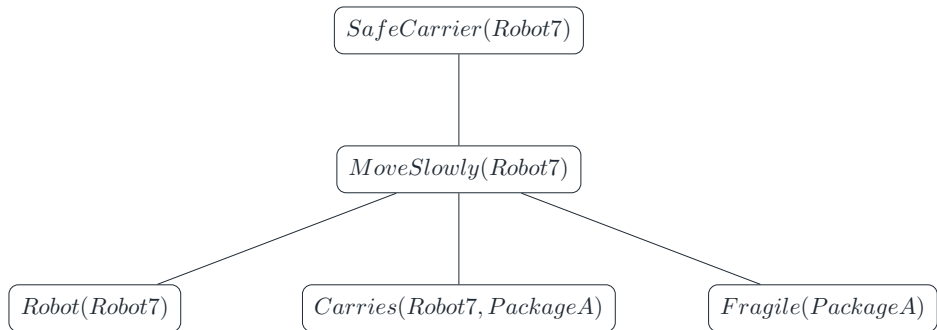
Fragile(PackageA)?

which is also a known fact.

Key idea

Backward chaining carries substitutions through the proof so later evidence can resolve earlier variables.

Backward Chaining Builds a Proof Tree



Key idea

Backward chaining is goal-directed search through rules, with unification determining which branches are applicable.

Forward vs. Backward Chaining in FOL

	Forward chaining	Backward chaining
Direction	Facts $\rightarrow$ consequences	Query $\rightarrow$ supporting premises
Style	Data-driven	Goal-driven
Uses unification to	Match rule premises	Match rule conclusions
Best when	Many consequences matter	A specific query matters
Search risk	Too many derived facts	Repeated / looping subgoals

Key idea

Both methods use the same logical machinery; they differ mainly in where the search begins.

Variable Names Are Local: Standardize Apart

Two rules might both use the variable name x :

$$Student(x) \rightarrow HasID(x)$$

$$Employee(x) \rightarrow HasBadge(x).$$

The two x symbols are independent placeholders, not automatically the same object. In inference procedures we often rename variables before combining rules:

$$x \mapsto x_1, \quad x \mapsto x_2.$$

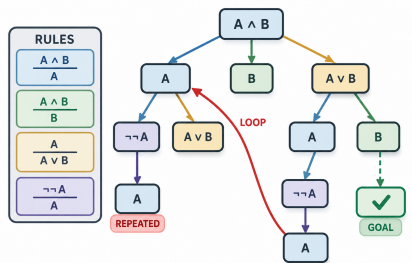
Key idea

Standardizing variables apart prevents accidental collisions between unrelated rule instances.

First-Order Inference Can Still Blow Up

Even compact rules may generate a large search space.

- ▶ Many facts may unify with the same premise.
- ▶ Multiple rule chains may prove the same goal.
- ▶ Recursive rules can create loops.
- ▶ Function symbols can generate endlessly deeper terms.



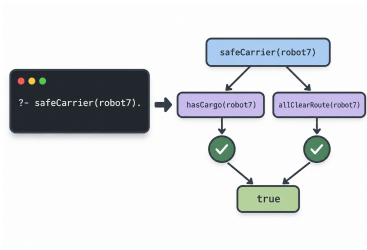
Key idea

FOL is compact to write, but inference still behaves like search—and search needs control strategies.

This Is the Core Idea Behind Logic Programming

Languages such as Prolog operationalize this style of reasoning:

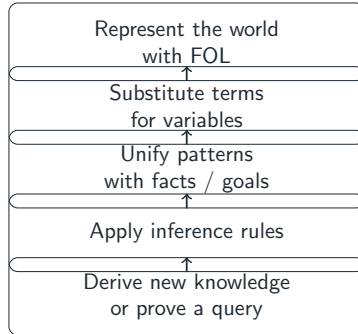
- ▶ facts are stored symbolically;
- ▶ rules contain variables;
- ▶ queries trigger goal-directed proof search;
- ▶ unification matches goals to facts and rule heads.



Key idea

In logic programming, computation itself can be viewed as proving that a query follows from a knowledge base.

The Whole Logic Unit in One Pipeline



Key idea

Knowledge representation gives the agent a language; unification and inference give that language computational force.

Takeaways

- ▶ A substitution maps variables to terms and instantiates general expressions.
- ▶ Unification finds a consistent substitution that makes two expressions identical.
- ▶ The most general unifier makes only the bindings required by the match.
- ▶ Generalized Modus Ponens combines unification with first-order rules.
- ▶ Forward chaining is data-driven; backward chaining is goal-driven.
- ▶ Both turn logical inference into a structured search problem over rules and substitutions.

Key idea

The machine does not “understand” that Socrates is mortal by intuition—it gets there by matching structure, binding variables, and applying a valid inference rule.